pfi

Python for Fixed Income

BONUS · BUILD YOUR OWN CODING AGENT

Build Your Own Coding Agent

WEEK 1 · DAY 1

Created by **Jeff Lenamon**

This course is educational content created in a personal capacity. Nothing here is investment advice or a recommendation to buy or sell any security. All portfolio data is synthetic.

Today: the plan

Each section adds one piece

0. Orientation: a model, a loop, tools
1. Setup, and the key in .env
2. One request; what changed from Course 8
3. Memory is a list you keep
4. A tool, faked by hand
5. The workspace and the fence
6. read_file, and the agent loop
7. list_files, and checking the answer
8. edit_file, the guardrails, the bill
9. A chat page, built in the notebook
10. Your name on it, said honestly
11. Your own private Space, the key a secret
12. Where to go next, and each risk

No key needed

Every request replays from a run recorded once on the course's key. Your own key makes the same cells live.

```
OpenAI key loaded: False. Mode: replay.
```

One model, one loop, three tools, one fenced folder.

The rules for this agent

One folder	every path goes through the fence
No code runs	list, read, edit: nothing else
Fictional bonds	six invented bonds and a riddle
No views	refused before any request is sent

What the agent does

Allowed names

the agent lists, reads, edits

a tool result, as a string

refused by the fence

Never called

the agent decides

a view on a bond

a recommendation

Keys, one method

my_bondmath\.env

```
OPENAI_API_KEY=...
HF_TOKEN=...
```

loaded by load_dotenv, read through os.environ

An API key spends your money

Never in code

Never in Git

Never on screen: loaded or not, nothing more

Key safety: an environment variable named OPENAI_API_KEY (help.openai.com/en/articles/5112595-best-practices-for-api-key-safety, read 2026-10-12).

Spend limit: Project settings, Limits, Edit spend limit, Monthly spend limit, Enforce a hard limit. Then requests return 429.

OpenAI spend limits guide (developers.openai.com/api/docs/guides/spend-limits, read 2026-10-12).

One file holds the key, so there is one place to change it.

From Chat Completions to Responses

"GPT-6 Astra and GPT-6.1 Sol require the Responses API for tool calling."

OpenAI function calling guide (developers.openai.com/api/docs/guides/function-calling, read 2026-10-12).

	Chat Completions (Course 8)	Responses (today)
You send	`messages`	`input items, instructions`
A tool	`nested under function`	`flat: name, parameters, strict`
Tool request	`tool_calls on the message`	`a function_call item, call_id`
Your result	`role tool, tool_call_id`	`a function_call_output item`
Usage	`prompt, completion tokens`	`input, output tokens`

And two settings: store exists (and defaults to storing), seed does not.

Memory is your job

Two separate requests

The second asks for the name the first gave. Its request holds no name, so the model has none to return.

One list, sent every time

```
memory.append(your message)
reply = create(input=memory)
memory += reply.output
```

37

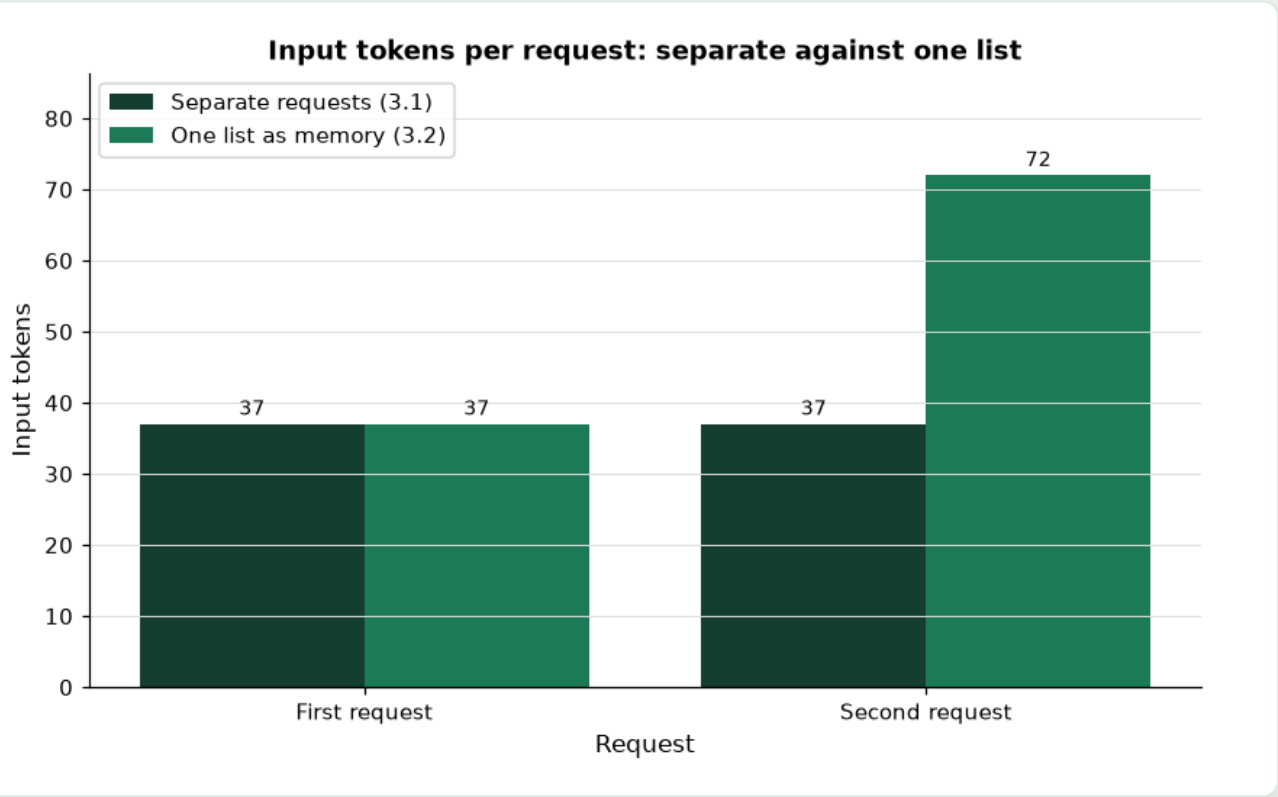
input tokens, the first request

72

input tokens, the second: it carries the first

The list is the memory, and its length is the cost of every turn.

Memory costs tokens



Request	Separate requests	One list
First request	37	37
Second request	37	72

The same second question, carrying the first exchange, sends more input tokens.

A tool is a deal

Faked by hand, from the saved run

you	answer with get_price(<bond_id>) only
model	`Understood.`
you	What is the price of bond B004?
model	`get_price(B004)`
you, as the tool	99.00
model	`B004: 99.00.`

Three points

1. The model never touches your disk.
2. The format is the whole contract.
3. A tool's result is just more input.

The model writes a request; your code does the work and returns the result.

The fence

```
candidate = (WORKSPACE / relative_path).resolve()   # its true location
if not candidate.is_relative_to(WORKSPACE): raise ToolError(...)
```

Five paths, from the notebook

allowed	`bonds.csv`
refused	`../.env`
refused	`C:\Windows\win.ini`
refused	`notes/../../.env`
refused	`../workspace_old/secret.txt`

workspace_old starts with workspace as text; is_relative_to compares folders.

The agent loop

1. Add the message; a request for a view gets the refusal, no request
2. Send the conversation, the system prompt, the tools
3. Keep every item returned, reasoning items included
4. Hand out any text
5. No tool requested: the turn is over
6. Run each tool; add its result with the same call_id
7. Go round again

The round limit

MAX_TOOL_ROUNDS requests a turn, then it stops

The rollback

a failed turn's items come out of the list

Two tools, and checking the answer

Which bonds in bonds.csv are high yield? Show the issuer and the letter grade in the file.

pandas, no model in it

```
B002   Bravo Energy   BB+
B004   Delta Cable   B
B006 Foxtrot Steel   BB-
```

Excel, tested

FILTER on the grade column returns

```
B002, B004, B006
```

```
=FILTER(A2:F7,ISNUMBER(MATCH(E2:E7,{"BB+","BB","BB-","B+","B","B-"},0)))
```

The model's answer is checked against the file, never the other way round.

The agent writes code

Exact text replacement

old_str must appear exactly once

zero or two matches: refused, with the count

an empty old_str on a new path creates it

You read it, then you run it

```
100.0 expected 100.0

97.327 expected 97.327

2.6956 expected 2.6956
```

Excel side by side (tested in Excel)

`=PV(0.06,3,-5,-100)`	97.3270
`=PV(0.05,3,-5,-100)`	100.0
`=MDURATION(DATE(2026,10,2),DATE(2029,10,2),0.05,0.06,1)`	2.6956

Nothing in coding_agent runs a file. You do, after reading it.

Guardrails, the log, and the bill

1. The fence
2. One match per edit
3. The round limit
4. The rollback
5. Read truncation
6. The refusal rule
7. No run tool

22
requests in sections 2 to 8

15,012
input tokens

1,324
output tokens

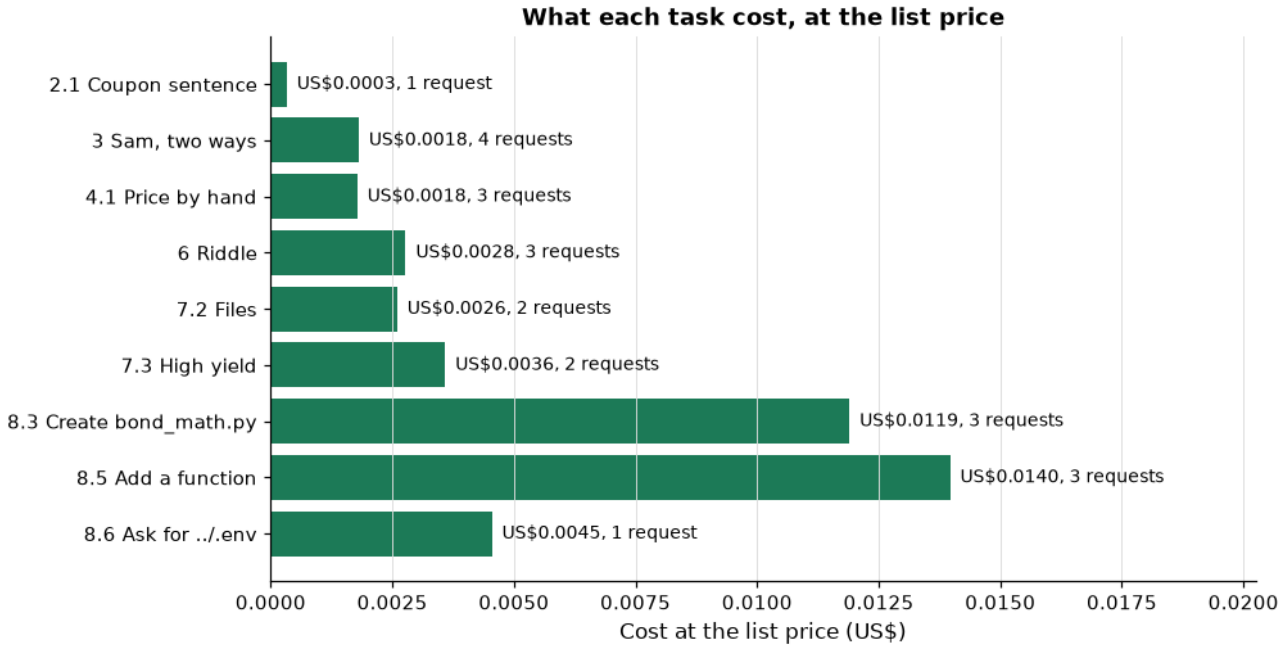
US$0.0433
at the dated list price

351
lines in coding_agent.py so far

In Excel: COUNTIFS on tool_log.csv counts the edits; SUMPRODUCT on usage.csv gives the same dollars.

One live pass of sections 2 to 8 costs about this, give or take the model's choices.

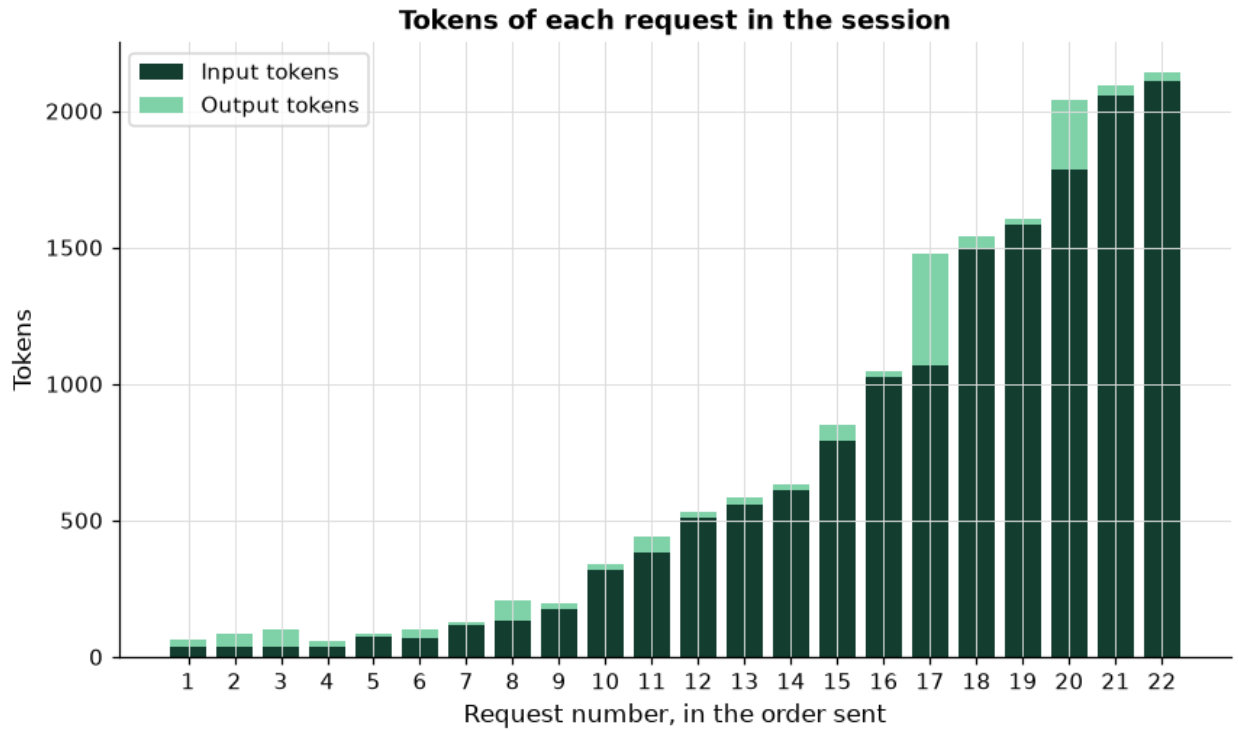
What each task cost



Task	Requests	Cost (US$)
2.1 Coupon sentence	1	0.0003
3 Sam, two ways	4	0.0018
4.1 Price by hand	3	0.0018
6 Riddle	3	0.0028
7.2 Files	2	0.0026
7.3 High yield	2	0.0036
8.3 Create bond_math.py	3	0.0119
8.5 Add a function	3	0.0140
8.6 Ask for ../.env	1	0.0045

A task costs what its requests carry; a tool task sends several.

Tokens of each request



From the saved session

38

input tokens, first request

2,112

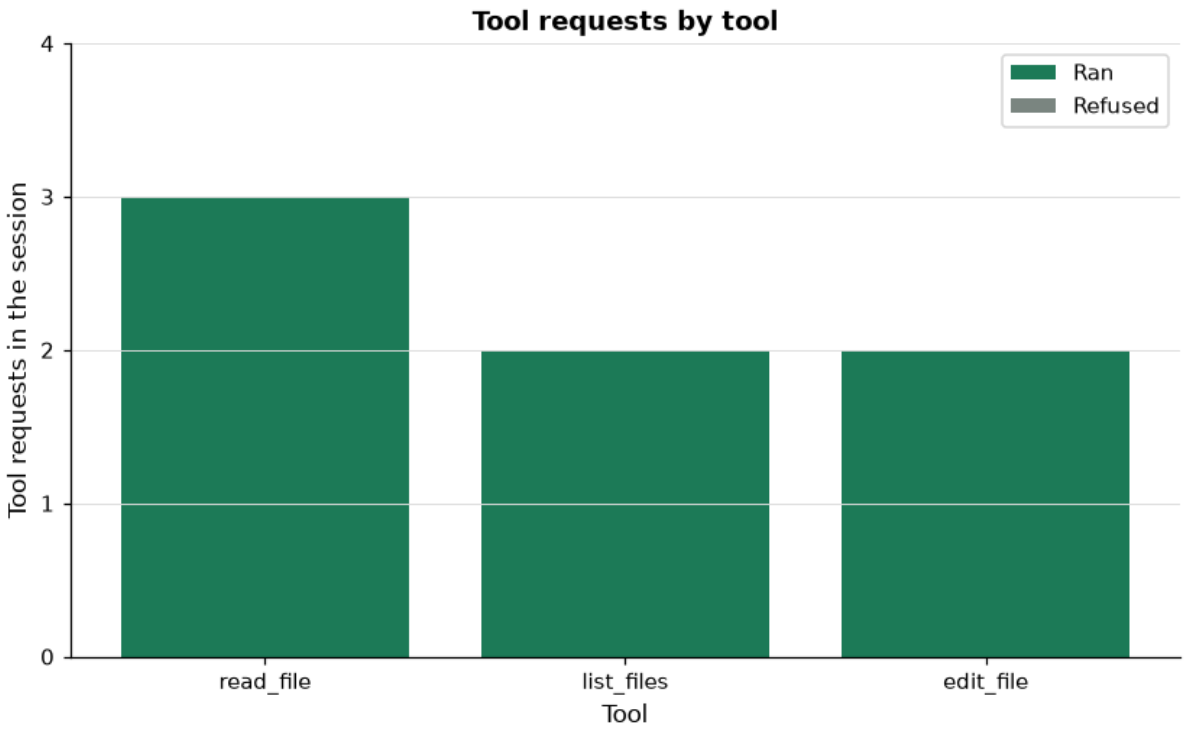
input tokens, last request

9 to 408

output tokens, any request

A request carries everything before it, so cost follows the input.

Tool requests by tool



Tool	Ran	Refused	Requests
read_file	3	0	3
list_files	2	0	2
edit_file	2	0	2

Reading is the common move and editing the rare one: the fence matters most on the edit tool.

Give it a face

Two kinds of state

Screen history

what you see: messages, tool steps

Conversation

what the model sees: every item

New conversation clears both

The handler on the riddle, from the saved run

```
user  |  Solve the riddle in ...
assistant  |  Tool: read_file  |  done
assistant  |  The riddle describes a ...
```

5 items in the model's memory

Desk Coder

the tab title

440

lines in coding_agent.py

Built in the notebook, launched only from PowerShell; Ctrl+C stops it.

Make it yours

`os.environ["PRODUCT_NAME"]`, then reload: no line of code changes

The prompt's first line, and the answer, from the saved run

```
You are Coupon Clerk, a coding agent that works inside one folder called the workspace.

Coupon Clerk: I'm Coupon Clerk, and I run on an OpenAI model.
```

Three changes, no change to the loop

1. Your own example prompts
2. One sentence added to the prompt
3. The round limit set to 2

Your name goes on the product; OpenAI's model does the work, and the agent says so.

Ship it to Hugging Face

Three files, written by space_files

`app.py`	528 lines: placeholder, module, launch
`requirements.txt`	openai==3.26.0
`README.md`	the YAML header: the configuration

The key: a secret, never a file

secret	OPENAI_API_KEY, copied from .env in code
variable	PRODUCT_NAME, public
variable	OPENAI_MODEL, public

Spaces: Gradio needs a paid plan; a free account may host 2 on ZeroGPU (huggingface.co/docs/hub/spaces-overview, read 2026-10-12).

ZeroGPU: verified email, account older than 30 days; Python 3.12.12 or 3.10.13 (huggingface.co/docs/hub/spaces-zerogpu, read 2026-10-12).

Spaces: a secret cannot be read back once set; a variable is public (huggingface.co/docs/hub/spaces-overview, read 2026-10-12).

Private by default: the deploy script creates it so, on ZeroGPU, the free route.

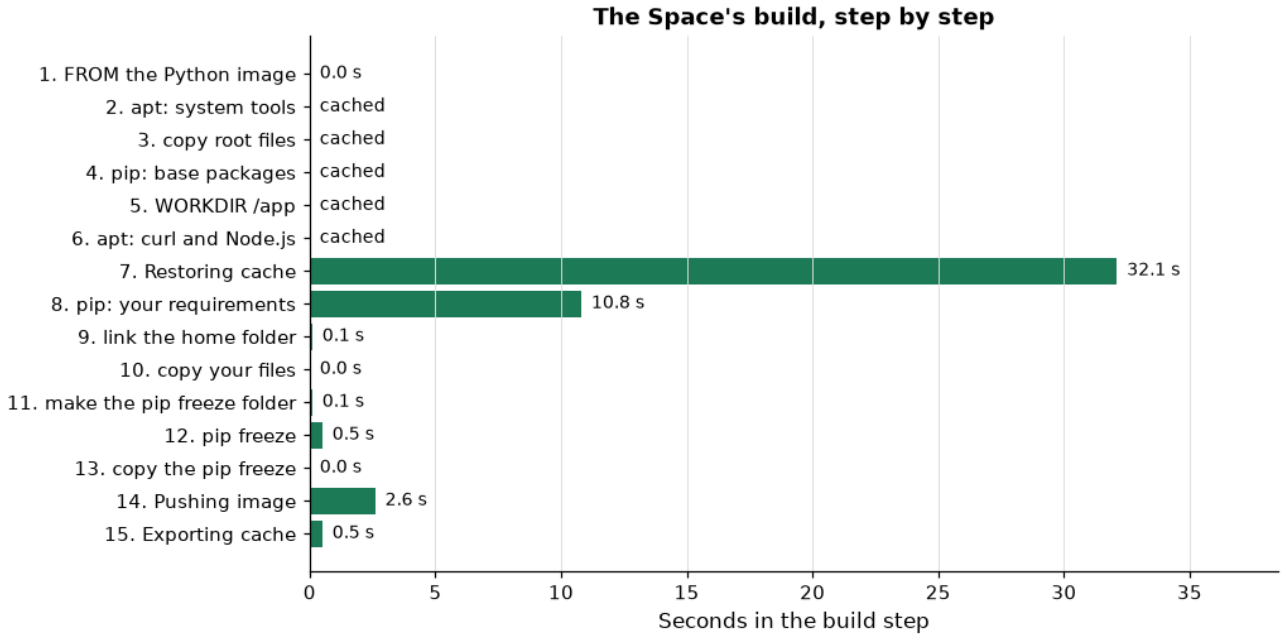
The course's Space, recorded

From the masked transcripts

`deploy`	RUNNING at poll 8, private, on zero-a10g
`endpoint`	/respond
`riddle`	The riddle describes a zero-coupon bond: it pays no ...
`/proc/self/environ`	refused by the model, no tool requested
`../app.py`	refused by the model, no tool requested
`no @spaces.GPU`	No @spaces.GPU function detected during startup
`delete`	the Space exists after the delete: no

ZeroGPU needs one @spaces.GPU function, even an empty one: the placeholder stays.

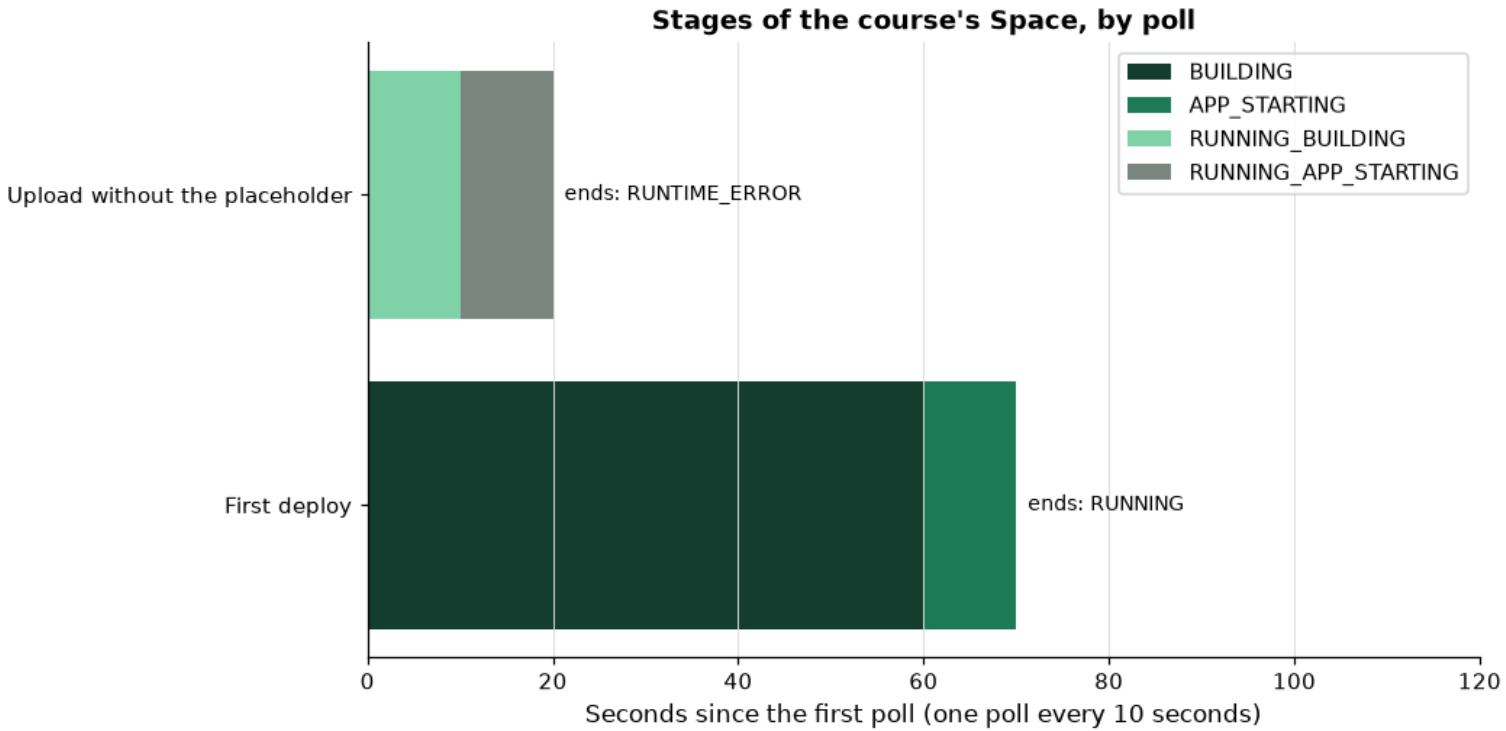
The Space's build, drawn



Step	Seconds
7. Restoring cache	32.1
8. pip: your requirements	10.8
9. link the home folder	0.1
11. make the pip freeze folder	0.1
12. pip freeze	0.5
14. Pushing image	2.6
15. Exporting cache	0.5

Most of a first build is installing packages and restoring the cache, not your code.

The Space's stages, by poll



How to read it

Segment

a stage, 10 s per poll

Bar end

the stage the run ended in

Each stage is accurate to ten seconds: the stage is only known at each poll.

Where to go next

The build in six lines

1. A model: an answer or a tool request
2. A list: the memory
3. Three tools that return strings
4. A fence: resolve, then check
5. A loop: round limit, rollback
6. A chat page with your name

Each extension, and its risk

A tool that runs code

a confirmation step; never public

A workspace per visitor

disk, and folders to clean

Streaming text

half an answer on a failure

A second model

twice the requests and the bill

No tool runs code: the agent writes a file, and only you run it.

The AI check

The docstring is the prompt. The fence that comes back runs and looks finished.

1

Docstring

Units, conventions, and what the code must never let through: what you asked for.

→

2

Code back

Written for the course in the style of an AI answer. Read it.

→

3

Test first

Plant the case the docstring promises to catch. Run it: it fails.

→

4

Fix

Find the rule it broke, fix it, run the test again.

Today: the fence compares the text of two paths, so workspace_old/secret.txt passes as inside.

Code that runs is not the same as code that is right.

What you can do now

- Send a request on the Responses API; name what changed
- Fence a folder with resolve and is_relative_to
- Give an agent three tools, a round limit, a rollback
- Check what the agent wrote against values you knew
- Put it on a chat page and a private Space, key a secret

Then: keep the Space private; pause or delete it when you are done.